

Projet Supply Chain

Analyse automatique des avis clients pour optimiser la satisfaction



Equipe:
Laurène ATTIA
Ilan SARFATI
Walide MATALLAH

Date de réalisation: 01/06/2025



Table des matières:

I- Introduction

- 1) Contexte du projet
- 2) Objectifs

II- Présentation des données

- 1) Contexte des données
- 2) Contenu du dataset

III- Exploration et préparation des données

- 1) Objectif
- 2) Exploration initiale
- 3) Traitement des données temporelles
- 4) Suppression de la variable «client»
- 5) Analyse de la variable « star »
- 6) Encodage des variables catégorielles
- 7) Traitement du texte
- 8) Gestion des valeurs manquantes

IV- Modélisation

- ❖ Objectif
- ❖ Séparation train/test
- ❖ Vectorisation TF-IDF
- ❖ Rééquilibrage SMOTE
- ❖ fonction d'évaluation des modèles
- ❖ Entraînement et évaluation des modèles
- ❖ Analyse des résultats

V- Amélioration des performances

- 1) Regroupement des classes et rééquilibrage avec SMOTE
- 2) Ajustement des hyper paramètres via Grid Search
- 3) Enrichissement du texte par extraction d'entités nommées (NER)
- 4) Identification du modèle le plus performant

VI- Conclusion et perspectives



I- Introduction

1. Contexte du projet

La **supply chain**, ou chaîne logistique, désigne l'ensemble des processus et des acteurs impliqués dans l'acheminement d'un produit ou service, depuis l'approvisionnement en matières premières jusqu'à la livraison finale au client. Elle englobe un vaste réseau d'activités interconnectées : planification, production, stockage, transport, gestion des flux d'information, contrôle qualité et logistique inverse.

Bien plus qu'un simple enchaînement d'étapes, la supply chain représente un **levier stratégique** pour les entreprises évoluant dans un environnement de plus en plus concurrentiel, internationalisé et orienté vers la personnalisation de l'offre. Elle vise à optimiser l'efficacité opérationnelle, la réactivité face aux demandes du marché et, in fine, la **satisfaction du client final**.

En entreprise, sa gestion est assurée par des professionnels spécialisés, capables de coordonner les flux physiques, informationnels et financiers pour garantir la **fiabilité, la qualité et la rentabilité** du processus global.

En aval de ce processus, il peut être intéressant d'évaluer la satisfaction client pour :

- ❖ **Étudier la qualité sur la supply chain** : problème de conception, livraison, prix non adapté, durabilité...
- ❖ **Étudier** si le produit/service correspond bien à l'**attente du marché**.
- ❖ **Synthétiser** les feedbacks, améliorations des clients.
- ❖ **Aider à la réponse** ou à la redirection des clients insatisfaits...

Pour de nombreux produits/services, la satisfaction des clients se mesure sur les commentaires, avis de sites dédiés (Trustpilot, site distributeur, twitter...).

2. Objectifs

Ce projet, réalisé dans le cadre de la formation **Datascientest**, vise à explorer et analyser un jeu de données d'avis clients afin de mieux comprendre les facteurs influençant la satisfaction et à proposer des outils automatisés d'aide à l'analyse.

Les principaux objectifs sont :

- ❖ **Prédire la satisfaction d'un client** : problème de régression (prédire le nombre d'étoiles).
- ❖ **Identifier les entités importantes** d'un message : localisation, nom d'entreprise...



- ❖ Extraire les propos du commentaire (problème de livraison, article défectueux...) : approche non supervisé
- ❖ Extraire de la réponse du fournisseur les propos du commentaire dans but d'essayer de les prédire uniquement avec le commentaire.

II- Présentation des données

1. Contexte des données

Les données utilisées pour ce projet proviennent de deux plateformes d'avis clients connues: **Trustpilot** et **Trusted Shops**. Elles se concentrent principalement sur les **retours des clients** concernant deux acteurs majeurs du secteur de la vente en ligne et des ventes privées : **Veepee** et **ShowRoom**.

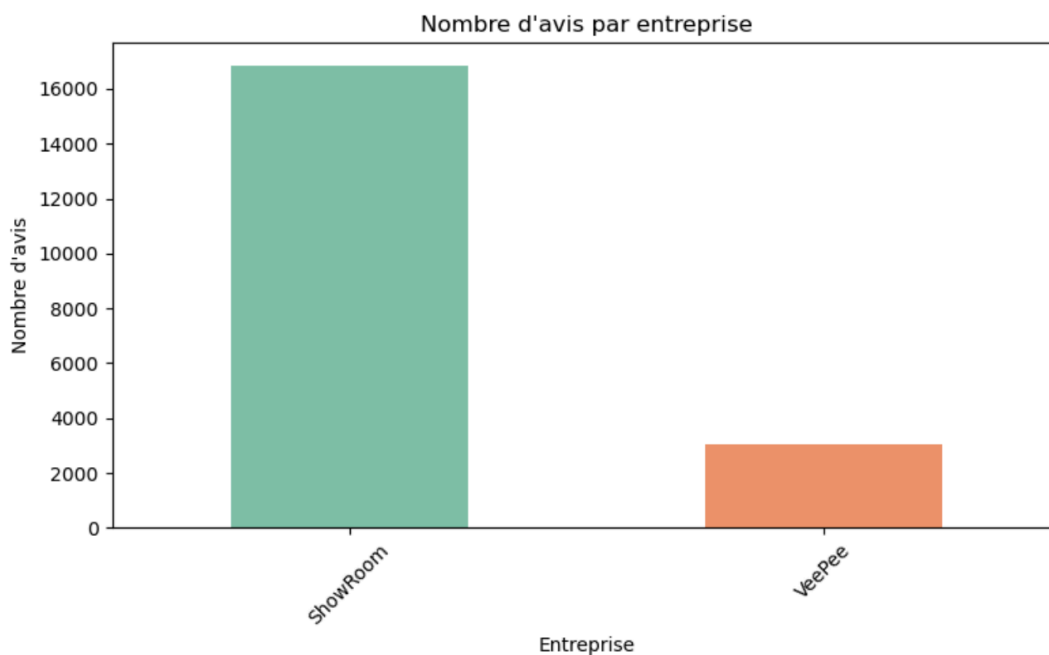


Figure 1: Graphique illustrant la répartition des données au sein du dataset

Le jeu de données est accessible via ce lien: [trust.csv](https://www.kaggle.com/datasets/veepee/trust-csv).



2. Contenu du dataset

Le jeu de données comprend plusieurs variables réparties en deux catégories principales:

- ❖ La **variable cible**, que nous cherchons à prédire.
- ❖ Les **variables explicatives** qui fournissent les informations nécessaires sur chaque observation.

Voici un tableau récapitulatif des variables du dataset:

Catégorie	Variable	Description
Variable cible (y)	star	Note de satisfaction attribuée par le client (de 1 à 5 étoiles).
Variables explicatives (X)	Commentaire	Avis textuel rédigé par le client.
	date	Date de publication de l'avis.
	client	Identifiant unique du client.
	reponse	Réponse apportée par l'entreprise à l'avis du client.
	source	Plateforme d'origine de l'avis (Trustpilot ou Trusted Shops).
	company	Nom de l'entreprise concernée (Veepee ou ShowRoom).
	ville	Ville renseignée par le client.
	maj	Date de dernière mise à jour éventuelle de l'avis.
	date_commande	Date à laquelle la commande a été passée.
	ecart	Écart entre la commande et la publication de l'avis.



III- Exploration et préparation des données

1. Objectif

L'objectif de cette première phase est de transformer les données brutes en un jeu de données propre, structuré et directement exploitable pour les prochaines étapes de modélisation. Elle comprend plusieurs étapes clés :

- ❖ Une **analyse exploratoire** des données,
- ❖ La gestion des **valeurs manquantes**,
- ❖ Le **traitement des variables temporelles et textuelles**,
- ❖ L'**encodage des variables catégorielles**,
- ❖ L'**élimination ou la transformation des variables** jugées peu pertinentes.

2. Exploration initiale

Le jeu de données comprend **19 863 avis clients** répartis sur **11 variables**.

Après le chargement des données, une première exploration a été menée afin d'analyser le **type de variables, leur distribution**, ainsi que la présence de **valeurs manquantes**.

Les **principaux constats** sont les suivants :

- ❖ La variable cible **star** (note attribuée par le client) est complète, avec une moyenne de 3,4/5, et présente une distribution exploitable.
- ❖ Certaines variables affichent un taux de valeurs manquantes très élevé : **maj** (19 858), **ville** (15 003), **client** (9 648), **date_commande** et **ecart** (13 177 chacune). Ces colonnes devront être évaluées selon leur utilité pour l'analyse.
- ❖ La variable **Commentaire**, qui constitue une source précieuse d'informations textuelles, est quasiment complète (seulement 29 valeurs manquantes).

3. Traitement des données temporelles

Lors de cette phase du projet, nous avons dans un premier temps, **converti les variables temporelles date, maj et date_commande au format DateTime**. En effet, cette conversion est nécessaire pour pouvoir **manipuler et analyser correctement les données temporelles**, comme le calcul d'intervalles entre dates, le tri chronologique, ou encore l'extraction de composantes spécifiques (année, mois, jour), ce qui facilite les analyses ultérieures et la modélisation.



Ensuite, nous avons créé plusieurs **nouvelles variables dérivées** (mois, jour, week-end, etc.) afin d'examiner leur corrélation avec la variable cible **star**.

Voici les résultats issus de l'étude de corrélation :

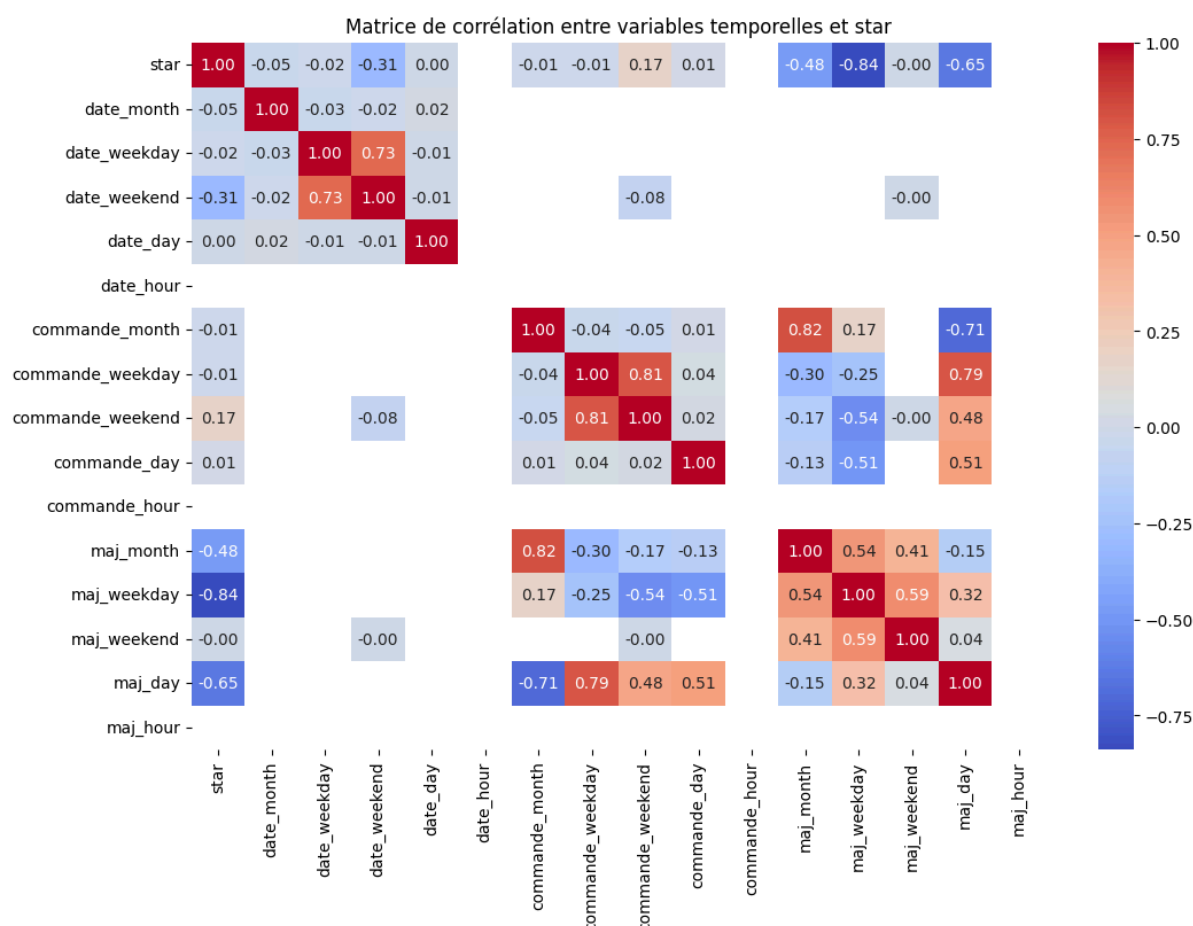


Figure 2: Matrice de corrélation entre les variables temporelles et la variable star

Analyse des résultats

- ❖ La majorité des **variables temporelles** présentent une corrélation quasi nulle avec la **variable cible star**, ce qui limite leur utilité pour la modélisation.
- ❖ Seule la variable **commande_weekend** affiche une corrélation positive modérée avec la cible (**coefficient : 0.16**), ce qui justifie sa conservation.
- ❖ Les variables horaires (**date_hour**, **commande_hour**, **maj_hour**) sont trop incomplètes pour être exploitées de manière fiable.



Décisions prises:

- ❖ Suppression des **variables temporelles** :
 - Trop de valeurs manquantes.
 - Corrélation négligeable avec la satisfaction client.
- ❖ Conservation de la variable **commande_weekend** :
 - Potentiellement informative.
 - Exploitable pour la phase de modélisation.

4. Suppression de la variable « client »

La variable **client**, composée d'**identifiants uniques**, n'apporte pas de valeur ajoutée à l'analyse ni à la modélisation. Pour simplifier le jeu de données et éliminer les informations non exploitables, nous avons décidé de la supprimer.

5. Analyse de la variable cible « star »

La variable **star** représente la **note de satisfaction** attribuée par les clients, sur une échelle de 1 à 5.

Distribution globale des notes:

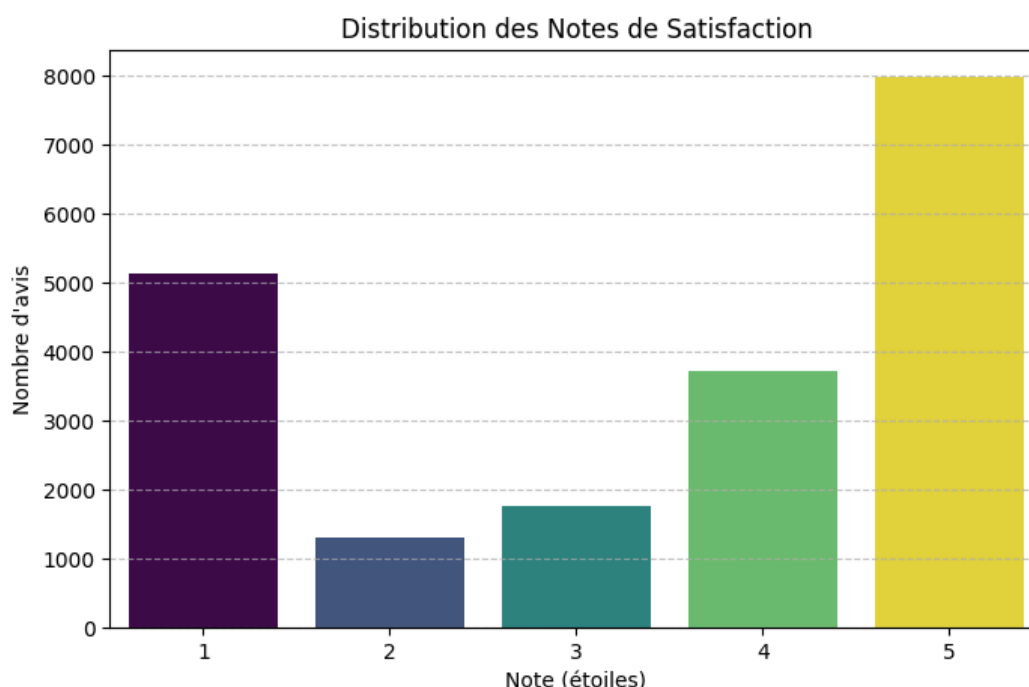


Figure 3: Répartition des notes au sein de la variable cible



Répartition des notes (en %) :	
star	
1	0.257866
2	0.065901
3	0.087952
4	0.186528
5	0.401752
..	

Figure 4: Répartition des notes (en %) au sein de la variable cible

L'analyse de la répartition des notes au sein de la variable cible met en évidence deux tendances marquées :

- ❖ **40 % des clients ont attribué 5 étoiles**, signalant une forte satisfaction,
- ❖ Tandis que **26 % ont donné minimale de 1 étoile**, traduisant un taux notable d'insatisfaction.

En effet, cette répartition reflète une **polarisation significative** dans les retours clients.

Distribution des notes par entreprise:

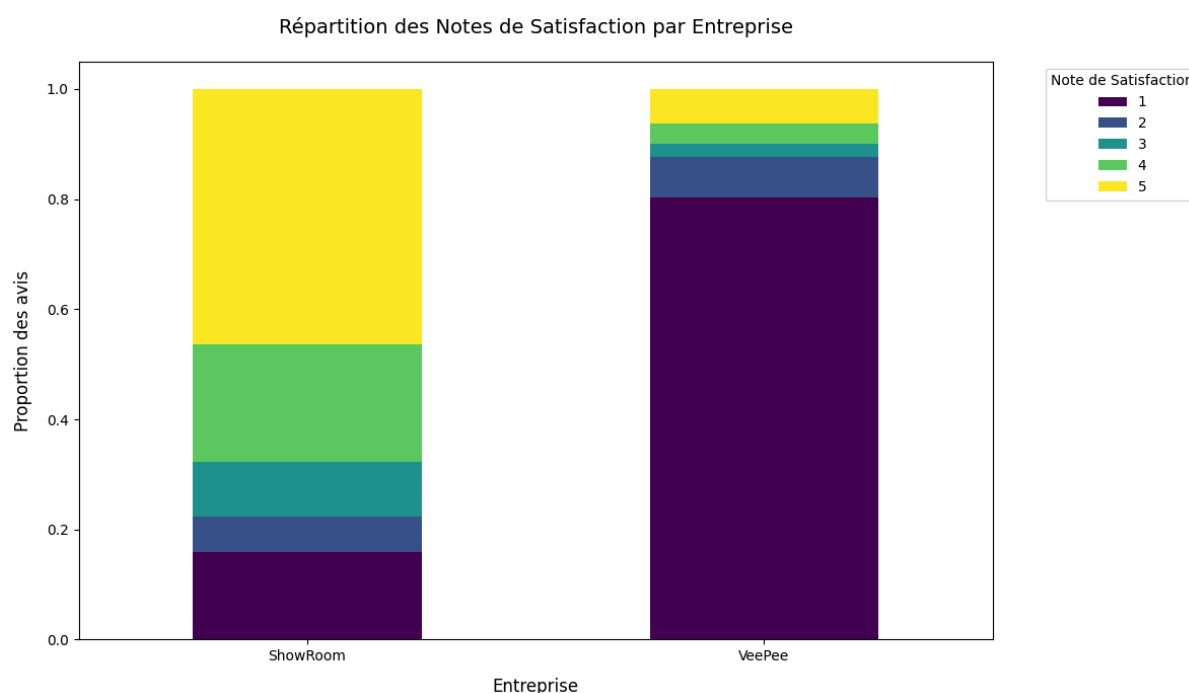


Figure 5: Répartition des notes par Entreprise au sein de la variable cible



Analyse:

- **ShowRoom** affiche une distribution plus équilibrée, avec une forte proportion de clients très satisfaits (note 5) et un faible taux de mécontentement (note 1). Cela témoigne d'une **expérience client globalement positive**.
- À l'inverse, **VeePee** présente une distribution déséquilibrée, dominée par les notes les plus basses, suggérant une **insatisfaction généralisée** et des **problèmes récurrents** dans l'expérience client.

Ces résultats montrent que **ShowRoom bénéficie d'une meilleure image auprès de ses clients**, tandis que **VeePee devra concentrer ses efforts sur l'amélioration de son service** pour faire face au fort mécontentement exprimé.

6. Encodage des variables catégorielles

Dans cette section, nous allons traiter l'encodage des variables catégorielles du jeu de données, afin de les rendre exploitables par les algorithmes de machine learning.

Voici les variables catégorielles présentes dans notre jeu de données:

- ❖ **company**: nom de l'entreprise (ShowRoom ou Veepee)
- ❖ **source**: plateforme d'origine de l'avis (TrustPilot ou TrustedShop)
- ❖ **ville**: ville du client

Encodage des variables company et source

Les variables **company** et **source** présentent un nombre restreint de modalités, ce qui rend l'encodage One-Hot particulièrement adapté. Nous avons donc utilisé la fonction **pd.get_dummies()** pour les transformer en variables indicatrices. Les colonnes générées, initialement de type booléen, ont ensuite été converties en entiers (1 ou 0) afin d'assurer leur compatibilité avec les algorithmes de machine learning.

À l'issue de ce traitement, les variables suivantes ont été intégrées au DataFrame : **company_ShowRoom**, **company_VeePee**, **source_TrustPilot** et **source_TrustedShop**.

Encodage de la variable ville

La variable **ville** comporte un grand nombre de modalités, rendant l'encodage One-Hot inapproprié. Un **encodage par fréquence** a donc été appliqué. Cependant, deux limites importantes ont été identifiées :



- Un **taux élevé de valeurs manquantes** (seulement environ 24 % des observations renseignées),
- Une **corrélation très faible avec la variable cible star** ($\approx -0,04$).

Compte tenu de sa faible contribution potentielle à la modélisation, la **variable ville** a finalement été supprimée du jeu de données.

Etat final du Dataset:

Notre jeu de données conserve désormais **8 variables**:

- ❖ Commentaire
- ❖ star
- ❖ reponse
- ❖ commande_weekend
- ❖ company_ShowRoom
- ❖ compagny_Veepee
- ❖ source_TrustPilot
- ❖ source_TrustedShop

7. Traitements des commentaires

Dans cette étape, nous avons procédé au nettoyage de la **colonne Commentaire** en appliquant les transformations suivantes :

- ❖ **Conversion** de tous les textes en **minuscules**,
- ❖ **Suppression de la ponctuation et des chiffres**,
- ❖ **Élimination des stopwords** en français,
- ❖ Création d'une **nouvelle colonne** contenant les commentaires nettoyés : **Commentaire_clean**.

Une fois cette préparation effectuée, nous avons analysé les termes les plus fréquents en générant un nuage de mots (*word cloud*).

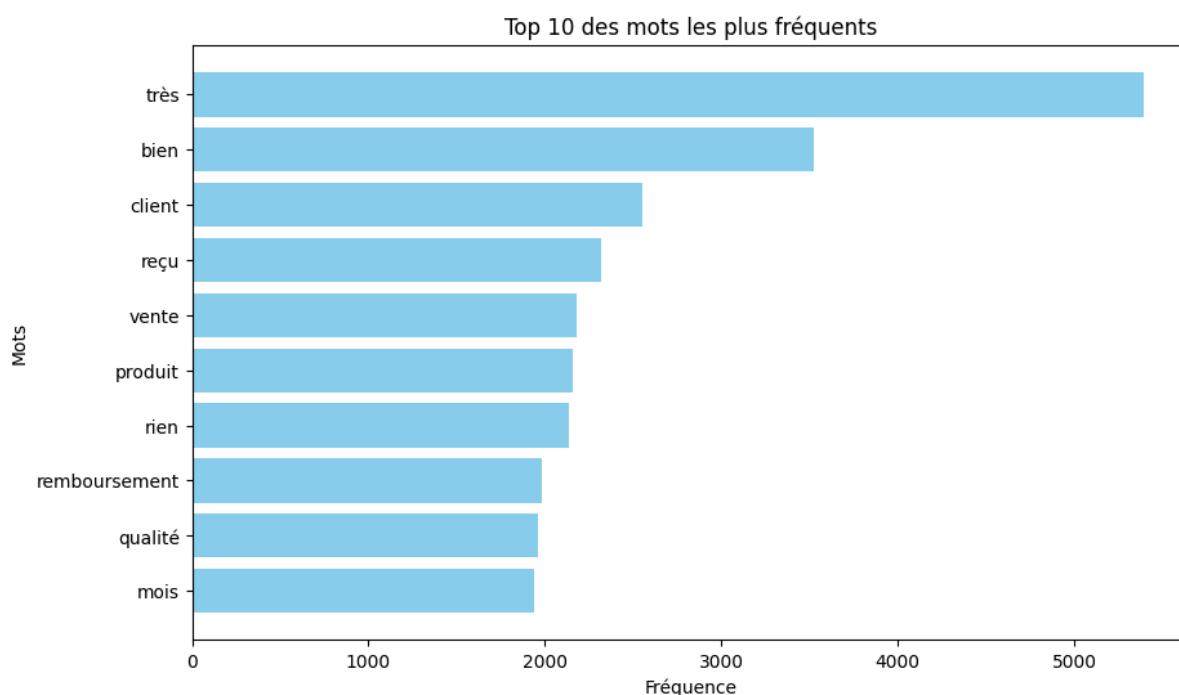


Figure 6: Top 10 des mots les plus fréquents

Ce nuage de mots met en évidence des préoccupations récurrentes liées au **produit**, au **service client** et à la **livraison**. Des termes révélateurs de frustration apparaissent également, tels que "**remboursement**" ou "**rien**", traduisant un certain mécontentement exprimé dans les avis.

8. Gestion des valeurs manquantes

Avant d'aborder la phase de modélisation, il est essentiel d'identifier et de traiter les valeurs manquantes afin de garantir la qualité du jeu de données.

L'analyse initiale a révélé les éléments suivants :

- ❖ 29 commentaires manquants dans la colonne **Commentaire**, représentant environ 0,15 % des données,
- ❖ Un taux élevé de valeurs manquantes dans la colonne **reponse**, à hauteur de 57 %,
- ❖ Aucune valeur manquante dans les autres variables.

Pour remédier à cela, deux mesures ont été mises en œuvre :



- ❖ **Suppression des lignes avec un commentaire absent**, car elles ne peuvent pas être exploitées dans le cadre de notre analyse textuelle,
- ❖ **Suppression de la colonne reponse**, jugée non indispensable pour notre objectif de prédiction de la satisfaction, en plus d'être largement incomplète.

À l'issue de ce nettoyage, **le jeu de données ne contient plus aucune valeur manquante**, ce qui garantit sa cohérence et sa fiabilité pour les étapes d'analyse et de modélisation à venir.

IV- Modélisation

1. Objectif

Cette phase a pour objectif de développer des modèles de machine learning capables de **prédire la note de satisfaction des clients** (de 1 à 5 étoiles) à partir du **contenu textuel des avis**. Pour cela, nous nous appuyons sur le **jeu de données nettoyé** lors de la phase précédente.

Cette phase comprend les étapes suivantes :

- ❖ **Séparation du jeu de données** en ensembles d'entraînement et de test, avec **stratification** sur la variable cible afin de préserver la distribution des notes.
- ❖ **Vectorisation des commentaires** à l'aide de la méthode **TF-IDF**, combinée à la suppression des stopwords pour améliorer la qualité des représentations textuelles.
- ❖ **Rééquilibrage des classes** via deux techniques de suréchantillonnage : **SMOTE** et **ADASYN**, afin de corriger le déséquilibre entre les différentes notes.
- ❖ **Entraînement et évaluation de plusieurs modèles classiques**, avec une **comparaison systématique des performances** à l'aide de métriques adaptées aux problèmes multiclasse.

Enfin, plusieurs **alternatives et pistes d'amélioration** sont explorées, notamment :

- ❖ Le **regroupement des classes** pour simplifier la tâche de classification,
- ❖ L'utilisation de la **recherche d'hyper paramètres (Grid Search)** pour optimiser les performances,
- ❖ Une approche combinant **reconnaissance d'entités nommées (NER)**, regroupement de classes, et suréchantillonnage via SMOTE.



2. Séparation train/test

Dans un premier temps, nous avons procédé à la **séparation des variables** du jeu de données:

- ❖ La **variable explicative X**, correspondant au **contenu textuel des avis** (colonne `Commentaire_clean`),
- ❖ Et la **variable cible y**, représentant la **note de satisfaction attribuée** (colonne `star`).

Le dataset a ensuite été divisé en deux sous-ensembles distincts:

- ❖ Un **jeu d'entraînement** (80 % des données), utilisé pour l'apprentissage des modèles,
- ❖ Et un **jeu de test** (20 %), destiné à évaluer les performances sur des données jamais vues.

Cette séparation a été réalisée à l'aide d'une **stratification basée sur la variable cible**, afin de **préserver la répartition des différentes classes de notes** dans les deux ensembles. Cette méthode permet d'assurer une représentativité équilibrée et d'éviter un biais potentiel dans l'évaluation des modèles.

Voici les dimensions des ensembles obtenus:

- ❖ **X_train** : 15 867 observations
- ❖ **X_test** : 3 967 observations
- ❖ **y_train** : 15 867 labels
- ❖ **y_test** : 3 967 labels

3. Vectorisation TF-IDF

Dans cette étape, les avis clients ont été transformés en représentations numériques à l'aide de la méthode **TF-IDF (Term Frequency - Inverse Document Frequency)**, afin de les rendre exploitables par les algorithmes de machine learning.

Cette technique permet d'évaluer l'**importance relative de chaque mot** dans un texte, en tenant compte à la fois de sa fréquence dans un document et de sa rareté dans l'ensemble du corpus. Elle permet ainsi de valoriser les termes significatifs tout en atténuant l'impact des mots trop fréquents et peu informatifs.

Les choix méthodologiques sont les suivants :



- ❖ **Suppression des stopwords** français pour éliminer les mots fréquents mais peu informatifs (ex. : “le”, “et”, “mais”),
- ❖ **Utilisation des n-grammes** (unigrams et bigrams) afin de mieux capturer le contexte local et les expressions fréquentes,
- ❖ **Limitation à 10 000 tokens** les plus fréquents pour réduire la complexité computationnelle.

Le vectoriseur a été **entraîné uniquement sur le jeu d'entraînement** pour éviter toute fuite d'information, puis appliqué au jeu de test en conservant le même vocabulaire et les mêmes dimensions.

4. Rééquilibrage SMOTE

Les graphiques ci-dessous présentent la **répartition des notes de satisfaction** dans le jeu d'entraînement, avant et après l'application de la méthode **SMOTE**.

Avant rééquilibrage, un déséquilibre important des classes est visible : les notes intermédiaires (notamment 2, 3 et 4 étoiles) sont significativement sous-représentées par rapport aux classes extrêmes (1 et 5 étoiles). Ce déséquilibre pourrait biaiser l'apprentissage des modèles, qui auraient tendance à favoriser les classes majoritaires.

Après application de SMOTE, la distribution des classes est uniformisée, chaque note disposant désormais d'un nombre similaire d'exemples, grâce à la génération d'échantillons synthétiques. Cette technique permet d'améliorer la capacité des modèles à reconnaître et prédire les classes minoritaires, renforçant ainsi la qualité et la généralisation des prédictions, notamment pour les notes moins fréquentes.

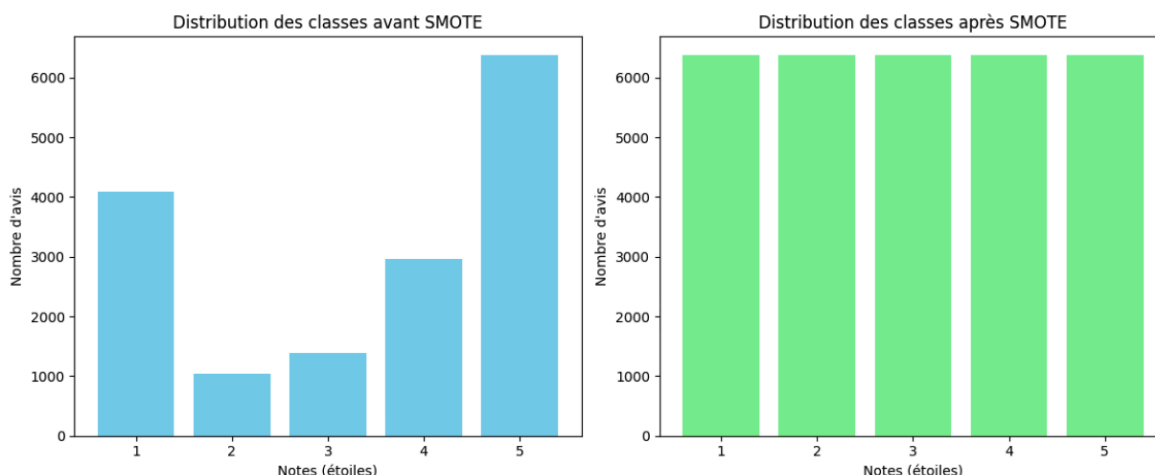


Figure 7: Répartition des notes avant/après l'application de SMOTE



5. Définition d'une fonction d'évaluation des modèles

Afin de standardiser l'entraînement, la prédiction et l'évaluation des différents modèles, nous avons créé une fonction dédiée. Cette fonction réalise les étapes suivantes :

- ❖ Entraînement du modèle sur les données d'entraînement vectorisées,
- ❖ Prédiction sur le jeu de test,
- ❖ Affichage des métriques classiques de classification (précision, rappel, F1-score) via un rapport détaillé,
- ❖ Visualisation de la matrice de confusion pour analyser les erreurs de classification,
- ❖ Calcul et retour d'un résumé des métriques principales (accuracy, macro-precision, macro-recall, macro F1-score), facilitant la comparaison entre modèles.

Cette approche permet d'**automatiser** et d'harmoniser l'**évaluation** afin de **comparer efficacement la performance des différents modèles**.

6. Entraînement et évaluation des modèles

Dans cette phase, nous avons sélectionné **six modèles de machine learning** classiques adaptés à la classification de données textuelles :

Modèle	Description
LogisticRegression	Modèle linéaire simple, efficace et rapide avec les données TF-IDF.
MultinomialNB	Classifieur naïf Bayes adapté aux données textuelles discrètes.
LinearSVC	SVM linéaire performant pour les tâches de NLP.
RandomForestClassifier	Ensemble d'arbres de décision capable de modéliser des relations complexes.
GradientBoostingClassifier	Méthode de boosting sur arbres, performante mais plus lente.
KNeighborsClassifier	Basé sur la proximité, simple, mais moins performant en haute dimension.

Chaque modèle a été **entraîné sur le jeu d'entraînement rééquilibré** à l'aide de **SMOTE**, puis **évalué sur le jeu de test**. Les performances ont été mesurées à l'aide de plusieurs indicateurs, notamment l'**accuracy**, la **précision**, le **rappel** et le **F1-score macro**. Ces résultats ont ensuite été rassemblés dans un tableau comparatif pour faciliter l'analyse.



7. Analyse des résultats

Le tableau ci-dessous illustre une synthèse comparative des performances obtenues pour chaque modèle.

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1-score
0	LogisticRegression	0.612554	0.492135	0.507653	0.494715
1	MultinomialNB	0.615831	0.485208	0.498074	0.485565
2	LinearSVC	0.612049	0.481134	0.492365	0.484038
3	RandomForestClassifier	0.644064	0.490936	0.484865	0.480087
4	GradientBoostingClassifier	0.570456	0.463841	0.459783	0.458935
5	KNeighborsClassifier	0.414923	0.434085	0.394636	0.370487

Figure 8: Tableau comparatif des performances des 6 modèles

Les **meilleurs résultats** ont été obtenus par les modèles **linéaires** (Logistic Regression, Multinomial Naive Bayes) et **à base d'arbres** (Random Forest), avec des **accuracies comprises entre 61 % et 64 %**, et des **F1-scores macro avoisinant 0.49**.

Cependant, les performances restent **modérées**, en particulier sur les **notes intermédiaires (2, 3, 4 étoiles)**, qui sont **systématiquement moins bien classées** que les notes extrêmes (1 et 5 étoiles).

Ces difficultés sont confirmées par l'analyse des matrices de confusion : on y observe un bon taux de reconnaissance des classes 1 et 5, tandis que les autres sont souvent mal différenciées.

La matrice de confusion suivante, issue du modèle Random Forest, illustre ces déséquilibres de prédiction :

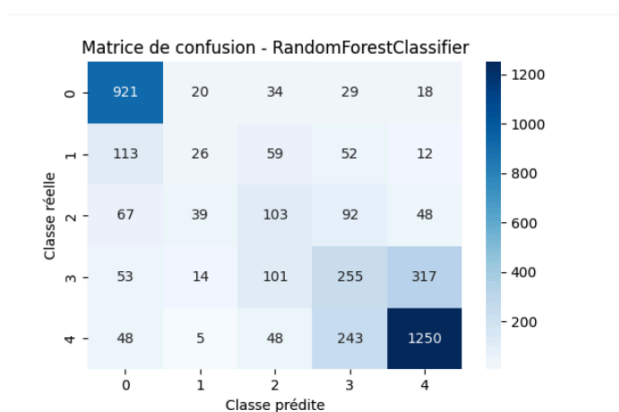


Figure 9: Matrice de confusion - Modèle RandomForestClassifier



Malgré l'utilisation de **SMOTE** pour rééquilibrer les classes, **les performances globales restent limitées**.

Cela indique que le simple suréchantillonnage ne suffit pas dans le cas de **données textuelles**. Il sera donc nécessaire d'explorer **d'autres approches complémentaires** afin d'améliorer les résultats, notamment sur les classes les moins représentées.

V- Amélioration des performances

1. Regroupement des classes et rééquilibrage avec SMOTE

La première méthode d'optimisation a consisté à **regrouper les classes proches** afin de simplifier le problème de classification. Les notes ont été reclassées de la manière suivante:

- ❖ Classe 1 : 1 étoile (négatif fort)
- ❖ Classe 2 : 2 et 3 étoiles (avis moyens)
- ❖ Classe 3 : 4 et 5 étoiles (positif fort)

Cette transformation vise à réduire l'ambiguïté entre les notes intermédiaires, qui étaient mal prédites précédemment. Après ce regroupement, un **rééquilibrage via SMOTE** a été appliqué pour obtenir une distribution plus homogène dans les classes du jeu d'entraînement.

Les résultats obtenus montrent une **amélioration nette des performances** :

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1-score	Rééquilibrage
0	RandomForestClassifier	0.815478	0.736684	0.754647	0.744657	SMOTE + regroupement
1	LogisticRegression	0.796824	0.729859	0.769376	0.741880	SMOTE + regroupement
2	MultinomialNB	0.788757	0.720793	0.765464	0.734443	SMOTE + regroupement
3	LinearSVC	0.791278	0.714831	0.744814	0.725545	SMOTE + regroupement
4	GradientBoostingClassifier	0.715906	0.699235	0.721605	0.683381	SMOTE + regroupement
5	KNeighborsClassifier	0.588606	0.653217	0.636975	0.576524	SMOTE + regroupement

Figure 10: Tableau comparatif des résultats après application du regroupement + SMOTE

Observations :

- ❖ La combinaison du regroupement des classes avec le rééchantillonnage SMOTE entraîne une **amélioration significative des performances**.



- ❖ Le modèle **RandomForest** se démarque avec la meilleure performance globale, atteignant 81,5 % de précision et un F1-macro de 74,5 %.
- ❖ Les performances des **modèles linéaires (régression logistique, MultinomialNB)** ont également fortement progressé grâce à ce prétraitement.
- ❖ Le GradientBoosting montre une amélioration notable, suggérant que la qualité initiale des données limitait ses performances.
- ❖ Bien que le KNN reste moins performant comparativement, il bénéficie également d'un gain après ces ajustements.

Conclusion :

- ❖ L'approche combinée « **SMOTE + regroupement des classes** » s'avère pertinente et efficace pour gérer ce jeu de données déséquilibré.

Pour illustrer ces résultats, voici la matrice de confusion du modèle RandomForestClassifier:

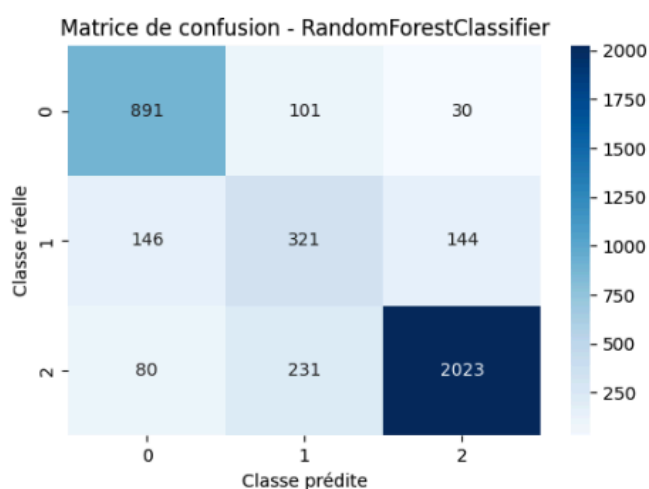


Figure 11: Matrice de confusion - Modèle RandomForestClassifier Optimisé(1)

2. Ajustement des hyper paramètres via Grid Search

Afin d'améliorer les performances des modèles, nous avons appliqué une **recherche aléatoire d'hyper paramètres (RandomizedSearchCV)** sur un sous-échantillon de **20 %** du jeu d'entraînement. Pour des raisons de temps de calcul, nous avons limité cette recherche aux trois meilleurs modèles identifiés précédemment : **Logistic Regression**, **Linear SVC** et **RandomForest**.



L'objectif était de maximiser le score **F1-macro**, mieux adapté aux jeux de données déséquilibrés que l'accuracy seule. Voici les meilleurs scores obtenus lors de la validation croisée :

- ❖ **Logistic Regression** : F1-macro = **0.6820**
- ❖ **Linear SVC** : F1-macro = **0.6745**
- ❖ **RandomForest** : F1-macro = **0.6051**

Une fois les meilleurs hyperparamètres identifiés, les modèles ont été évalués sur le jeu de test complet. Le tableau ci-dessous présente les performances du modèle **Logistic Regression**, qui s'est avéré le plus performant.

Matrice de confusion – Logistic Regression

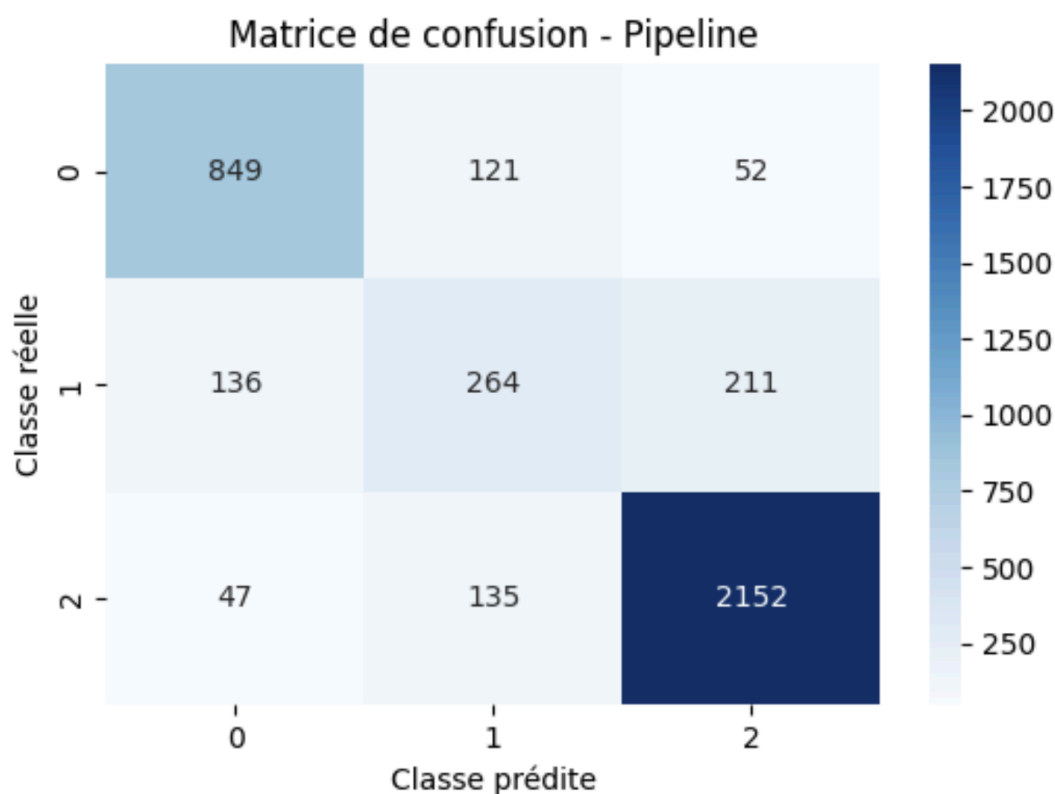


Figure 12: Matrice de confusion - Modèle RandomForestClassifier Optimisé(2)

Rapport de classification – Logistic Regression



=== Résultats pour Pipeline ===				
	precision	recall	f1-score	support
1	0.82	0.83	0.83	1022
2	0.51	0.43	0.47	611
3	0.89	0.92	0.91	2334
accuracy			0.82	3967
macro avg	0.74	0.73	0.73	3967
weighted avg	0.81	0.82	0.82	3967

Figure 13: Rapport de Classification - Modèle RandomForestClassifier Optimisé(2)

Bien que l'optimisation des hyperparamètres via Grid Search ait permis d'améliorer les performances des modèles, notamment en ajustant leur capacité à généraliser, **cette méthode ne parvient pas à compenser le déséquilibre des classes**. En particulier, la classe intermédiaire (classe 2) reste difficile à prédire, comme le montre son faible rappel.

Ainsi, pour obtenir de meilleurs résultats, notamment en termes de score F1-macro, il est indispensable d'associer l'optimisation des paramètres à des techniques de rééquilibrage des classes, telles que SMOTE et le regroupement des classes. Une approche combinée permettrait d'optimiser à la fois les performances des modèles et leur capacité à traiter les déséquilibres structurels des données.

3. Enrichissement du texte par extraction d'entités nommées (NER)

Dans cette dernière approche, nous avons cherché à **enrichir le contenu textuel des commentaires grâce à l'extraction d'entités nommées (Named Entity Recognition - NER)** en utilisant le **modèle SpaCy**. Cette technique permet d'ajouter au texte les entités détectées (noms de personnes, lieux, organisations, etc.) afin de renforcer la représentation sémantique des commentaires.

Une fois les entités extraites et concaténées au texte nettoyé, nous avons repris le pipeline précédent en y appliquant :

- ❖ un **regroupement des classes** (1, 2-3, 4-5),
- ❖ un **rééquilibrage des classes** avec **SMOTE**,
- ❖ puis l'évaluation de plusieurs modèles sur ce texte enrichi.



Voici les résultats obtenus:

=== Résultats FINAUX avec NER (entités) + regroupement de classes + SMOTE ===				
	Model	Accuracy	Macro Precision	Macro Recall \
0	LogisticRegression	0.798336	0.730672	0.769574
3	RandomForestClassifier	0.813461	0.732906	0.749955
1	MultinomialNB	0.787245	0.719363	0.763690
2	LinearSVC	0.791530	0.714053	0.741329
4	GradientBoostingClassifier	0.720192	0.700718	0.725097
5	KNeighborsClassifier	0.594656	0.652141	0.647147
Macro F1-score				
0		0.742832		
3		0.740408		
1		0.732790		
2		0.724160		
4		0.686843		
5		0.583014		

Figure 14: Tableau comparatif des performances après application de la méthode NER

Observations:

- ❖ L'ajout des entités nommées améliore globalement les performances, notamment pour la classe intermédiaire (2), difficile à prédire jusqu'ici.
- ❖ La **Logistic Regression** obtient le meilleur **score F1-macro**, bien qu'en accuracy, le **RandomForest** soit légèrement supérieur.
- ❖ Tous les modèles (sauf KNN) profitent de cette représentation enrichie, ce qui confirme l'intérêt d'intégrer des éléments sémantiques au texte.
- ❖ Le **KNN** reste en retrait malgré l'enrichissement, indiquant qu'il s'adapte mal aux données vectorielles textuelles enrichies.

VI- Conclusion et perspectives

Parmi toutes les approches testées, la méthode combinant **SMOTE** et **regroupement des classes** reste, à ce stade, la plus efficace pour traiter le déséquilibre du jeu de données. Elle permet une nette amélioration du **score F1-macro**, en particulier sur la classe intermédiaire difficile à modéliser.

En effet, l'enrichissement du texte via la **reconnaissance d'entités nommées (NER)** apporte un gain supplémentaire, mais les résultats montrent que l'impact reste secondaire comparé à



l'effet du rééquilibrage et du regroupement. Cette stratégie pourrait toutefois être explorée plus en profondeur pour de futures améliorations.

Par ailleurs, après avoir testé plusieurs modèles classiques de classification (LogisticRegression, MultinomialNB, LinearSVC, RandomForestClassifier, GradientBoostingClassifier, KNeighborsClassifier) sur les données textuelles enrichies, équilibrées avec SMOTE et avec un regroupement des classes (1, 2&3, 4&5), le **RandomForestClassifier s'est révélé être le modèle le plus performant.**

En effet, il atteint la **meilleure précision globale (environ 81.5%)** ainsi qu'un score **F1-macro élevé (environ 74.5%)**, démontrant une bonne capacité à gérer le déséquilibre des classes et à généraliser sur les données de test.

Pour valider concrètement la pertinence de ce modèle, nous avons réalisé des tests de prédiction sur des commentaires réels extraits du jeu de test. Voici un exemple illustratif :

Exemples de résultats obtenus :

- **Commentaire** : très bon produit
Classe prédite (regroupée) : 3
Correspondance possible : Note 4 ou 5
- **Commentaire** : sandales conformes présentation parfait
Classe prédite (regroupée) : 3
Correspondance possible : Note 4 ou 5
- **Commentaire** :
Assez régulièrement, Showroomprivé déçoit malheureusement. Leur service se dégrade avec un manque de professionnalisme, notamment au niveau des livraisons. Par exemple, plusieurs colis ont été laissés sous des boîtes aux lettres sans passage du livreur, ou des commandes annulées au dernier moment alors qu'elles étaient soi-disant déjà transmises. Je commence à me demander s'il est encore pertinent de continuer à commander chez eux.
Classe prédite (regroupée) : 1
Correspondance possible : Note 1

Ces tests confirment que le RandomForestClassifier, entraîné avec SMOTE et regroupement des classes, est capable de fournir des prédictions cohérentes et robustes. Cette méthode constitue la meilleure approche pour la classification des notes à partir des commentaires clients dans ce projet.

